Session 5 · Deck 5.B4

# Determining the Coefficients, and the Worked Example

Zhigang Feng

Thursday, October 22, 2026

© Zhigang Feng Redistribution prohibited without permission

*Build it on paper → solve it on the machine → push it to the frontier with AI*

# Where We Are

you are here: 5.B4 ▼

| S1 ✓ | S2 ✓ | S3 ✓ | S4 ✓ | S5 | S6 | S7 | S8 | S9 | S10 |
|---|---|---|---|---|---|---|---|---|---|
| Sep 24 | Oct 1 | Oct 8 | Oct 15 | Oct 22 | Oct 29 | Nov 5 | Nov 12 | Nov 19 | Dec 3 |
| Intro | HA · OLG | Num. DP | Numerics | Numerics | HA classic | ML | RL | Text | Agents |
| Growth | Policy | Python | I | II | The wall | Deep nets | HA + DL | LLMs | Projects |
| trunk | branches | the machine · classical methods | | | the wall | new tools | | new data | colleagues |

*Build it on paper → solve it on the machine → push it to the frontier with AI*

**Block B — projection:** 5.B1 the residual · 5.B2 choosing a basis · 5.B3 many dimensions and finite elements · **5.B4** coefficients and the worked example.

© Zhigang Feng
Redistribution prohibited without permission

# Roadmap: Four Parts

**The basis is chosen. Now 5.B1's second question, what "small" means; then the growth model a fourth way.**

| Part | Topic | The question it answers |
|---|---|---|
| 1 | Weighted residuals | How does "make $R$ small" become a system of equations? |
| 2 | Six weight functions | What are the choices, what does each mean, and which to use? |
| 3 | The worked example | How is the Block A model solved by Chebyshev collocation, step by step? |
| 4 | One model, four methods | Which of the four solvers wins, and when does the ranking reverse? |

© Zhigang Feng Redistribution prohibited without permission

By the end: you will be able to write the $56$ equations the worked example solves, say why they are zero at the nodes and small elsewhere, and defend the choice of orthogonal collocation.

**"Small" means zero in a weighted-integral sense: choose $n+1$ weight functions, and you have $n+1$ equations for $n+1$ coefficients.**

Given weight functions $w_i : \Omega \to \mathbb{R}^m$, $i = 0, \ldots, n$, require

$$\int_\Omega \underbrace{w_i(x)}_{\text{weight } i} \underbrace{R(x;\theta)}_{\text{residual}} \, dx \;=\; 0, \qquad i = 0, \ldots, n.$$

This is 5.B1's metric $\rho$ made concrete: $\rho = 0$ when all $n+1$ weighted integrals vanish, $1$ otherwise. Hence the other name, **weighted residual methods**.

A rather intractable functional equation has become a **standard nonlinear system**: Newton for small problems, conjugate gradient for big ones (4.A2). The remaining issue is the choice of $w_i$ — Part 2.

**Idea: what the weight is for**

The function being approximated may change rapidly in one part of the domain and slowly elsewhere. The weight says where accuracy matters: more weight where the function varies most, and the solve spends its coefficients there — as in weighted least squares.

A natural weight is the **basis itself**: chosen to match the function's features, it already emphasises the right regions. That choice is **Galerkin**.

© Zhuoqing Yang Redistribution prohibited without permission

**Weight by the residual's own sensitivity to each coefficient. That is the first-order condition of "minimise the squared residual", so it is the regression answer.**

Take $w_i(x) = \dfrac{\partial R(x;\theta)}{\partial \theta_i}$.

This is not arbitrary: it is the solution of the variational problem

$$\min_{\theta} \int_{\Omega} R^2(x;\theta)\, dx,$$

whose first-order condition is exactly $\int_{\Omega} \partial_{\theta_i} R \cdot R\, dx = 0$ for each $i$. Mathematically it is a standard regression problem in econometrics.

**Attractive:** it always generates **symmetric** matrices — convenient in proofs, and exploitable by algorithms that are faster and use less memory.

**Pitfall**

Least squares may lead to **ill-conditioning** and systems that are complicated to solve numerically: squaring the residual squares the sensitivity to rounding too.

© Zhigang Feng — Redistribution prohibited without permission

# Weights II and III: Subdomain, and Moments

**Subdomain asks the residual to average to zero on each piece of the domain. Moments ask its first $n+1$ moments to vanish, and are to be avoided.**

**Subdomain.** Split $\Omega$ into $n+1$ pieces $\Omega_i$ and take step functions,

$$w_i(x) = \begin{cases} 1 & x \in \Omega_i \\ 0 & \text{otherwise,} \end{cases} \qquad \text{so} \qquad \int_{\Omega_i} R(x;\theta)\,dx = 0.$$

The residual **averages to zero on each piece**; it may be large inside a piece as long as it cancels.

**Moments.** Take $\{1, x, x^2, \ldots, x^n\}$ as weights:

$$\int_{\Omega} x^i R(x;\theta)\,dx = 0, \qquad i = 0, \ldots, n.$$

The residual's moments of order $0$ to $n$ are zero.

**Idea**

Subdomain solves small, local conditions instead of one global one; the first moment condition just says the residual integrates to zero, and the rest constrain its shape.

**Pitfall**

Moments, widely used in engineering, work for low $n$ ($2$ or $3$). For higher orders the numerical performance is very poor: high powers of $x$ are highly collinear — 5.B2's argument — and rounding error takes over. **Moments are to be avoided as weight functions.**

© Zhigang Feng · Redistribution prohibited without permission

# Weight IV: Collocation

**A Dirac delta is a weight that looks at one point only. Collocation asks the residual to vanish where you look, and is silent where you do not.**

Take $w_i(x) = \delta(x - x_i)$, the Dirac delta: zero everywhere except at $x_i$, with $\int \delta(x - x_i)\, dx = 1$. Since $\int \delta(x - x_i) R(x)\, dx = R(x_i)$, the conditions collapse to

$$R(x_i; \theta) = 0, \qquad i = 0, \ldots, n,$$

at the $n + 1$ **collocation points**. The residual is exactly zero there and unconstrained in between — 5.B1's picture.

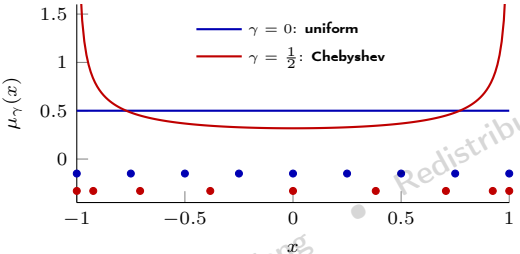
Simple to compute: the "integral" is one function evaluation, which is especially attractive with strong nonlinearities. But **where you look** is the whole design: equi-spaced points are the obvious choice and, by 4.B1's Runge example, the wrong one.

| | **Collocation** | **Subdomain** |
|---|---|---|
| Weight | $\delta(x - x_i)$ | step function |
| Residual | zero at points | zero on average per piece |
| Evaluation | pointwise | an integral each |
| Cost | lower | higher |
| Best for | nonlinear, spectral | smooth, PDE-like |
| Limitation | misses between points | integration overhead |

© Zhuoran Yang · Redistribution prohibited without permission

# Picking the Collocation Points: A Density

**A systematic rule: choose a density on $[-1, 1]$ and place the points at its quantiles. Points go where the density is high.**




Dots: the 9 points ($n = 8$) each density places.

A one-parameter family of densities,

$$\mu_\gamma(x) = \frac{\Gamma(\frac{3}{2} - \gamma)}{\sqrt{\pi}\,\Gamma(1-\gamma)}\,(1-x^2)^{-\gamma}, \quad 0 \le \gamma < 1,$$

with the points as its quantiles: $\int_{-1}^{x_j} \mu_\gamma = j/n$, $j = 0, \ldots, n$.

- $\gamma = 0$: uniform, so the points are **equi-spaced**.
- $\gamma = \frac{1}{2}$: $1/(\pi\sqrt{1-x^2})$, the Chebyshev weight; the points are $x_j = -\cos(\pi j/n)$.
- Larger $\gamma$: more mass near $\pm 1$, so the points crowd the ends.

© Zhigang Feng Redistribution prohibited without permission

# Weights V and VI: Orthogonal Collocation, and Galerkin

**Orthogonal collocation puts the points at the roots of the next orthogonal polynomial. Galerkin makes the residual orthogonal to every basis function.**

**Orthogonal collocation**, in three steps:

1. take an **orthogonal** family as the basis (Legendre, Chebyshev);
2. with $n+1$ coefficients, compute the $n+1$ **roots of** $T_{n+1}$;
3. impose $R(x_j;\theta)=0$ at those roots.

The roots cluster at the endpoints, capturing rapid changes there and avoiding Runge's phenomenon. Its performance is surprisingly good.

**Galerkin** (or Rayleigh–Ritz): $w_i=\psi_i$,

$$\int_\Omega \psi_i(x)\,R(x;\theta)\,dx=0, \qquad i=0,\dots,n.$$

**Idea: what Galerkin means**

The residual has **no component the basis could still represent**: whatever is left is invisible in this function space, so the basis has given everything it can.

If the basis is complete, the Galerkin solution converges **pointwise** to the true $d(x)$ as $n\to\infty$. It is highly accurate and robust, and *difficult to code*: every condition is an integral of a nonlinear function.

© Zhirang Zone · Redistribution prohibited without permission

# Six Weights, One Model, One Basis

**All six on Block A's model with the same Chebyshev basis: every method lands within one order of magnitude of every other, and the cheapest is the most accurate.**

Chebyshev degree $6$ in $k$ for each of $7$ shock states: $49$ unknowns in every row. Only the weight changes. Error is the largest Euler residual on test points away from the nodes.

| **Weight** | $\max \lvert\mathcal{E}\rvert$ | **sec** |
|---|---|---|
| orthogonal collocation | $\mathbf{10^{-5.78}}$ | 0.11 |
| least squares | $10^{-5.61}$ | 0.80 |
| Galerkin | $10^{-5.55}$ | 0.20 |
| moments | $10^{-5.55}$ | 0.15 |
| collocation, equi-spaced | $10^{-5.39}$ | 0.11 |
| subdomain | $10^{-5.05}$ | 0.14 |

**Ledger** `collocation`. **Warm-started from the first-order perturbation solution.**

**Use orthogonal collocation.** It is the most accurate here, ties for fastest, and evaluates no integrals at all. The worked example does.

Least squares is seven times slower for no gain — the ill-conditioning of Weight I, showing up as a solver that will not converge its last digits. Galerkin and moments agree because $\{1, \ldots, x^6\}$ and $\{T_0, \ldots, T_6\}$ span the same space.

© Zhigang Feng
Reproduction prohibited without permission

**The stochastic growth model of 3.A4, 4.B4 and 5.A3, now by Chebyshev collocation. Four commitments, made explicitly.**

| | |
|---|---|
| **3.A4** | grid VFI, everything crude |
| **4.B4** | grid VFI, Rouwenhorst + interpolation + golden section + Howard |
| **5.A3** | perturbation around $k^*$ |
| **5.B4** | **Chebyshev collocation on a domain** |

The unknown is the **consumption policy**, a Chebyshev polynomial in $k$ for each of the seven shock states; the coefficients zero the Euler residual at the Chebyshev nodes.

**The commitments**

**Basis:** Chebyshev in $k$, degree 7 (5.B2).

**Domain:** $k \in [0.5k^*, 1.5k^*]$ — chosen, and it matters.

**Shock:** the Rouwenhorst 7-state chain of 4.B4, so the comparison is like for like.

**Projection:** orthogonal collocation (Part 2).

© Zhirang Zhou • Redistribution prohibited without permission

# Steps 1 and 2: The Basis, and the Residual

**One polynomial per shock state, on capital mapped to $[-1, 1]$. The residual is the unit-free Euler equation, now the equation being solved rather than a diagnostic.**

**Step 1, the basis.** Map capital to $[-1, 1]$ with $\psi(k) = 2\frac{k - k_{lo}}{k_{hi} - k_{lo}} - 1$ (5.B2). Productivity takes seven values, so write **one polynomial per shock state**:

$$\hat{c}_j(k) = \sum_{i=0}^{7} \theta_{ji}\, T_i\big(\psi(k)\big), \qquad j = 1, \ldots, 7.$$

**Step 2, the residual.** Given $\theta$ and a point $(k, z_j)$: consume $\hat{c}_j(k)$, carry the rest forward as $k'$, and average tomorrow's marginal return over the row $\Pi_{j\cdot}$:

1: **function** RESIDUAL$(\theta, k, j)$
2: $\quad c \leftarrow \hat{c}_j(k)$
3: $\quad k' \leftarrow e^{z_j} k^{\alpha} + (1 - \delta)k - c$, clipped to $[k_{lo}, k_{hi}]$
4: $\quad E \leftarrow \sum_{l=1}^{7} \Pi_{jl} \left(\alpha e^{z_l} k'^{\alpha - 1} + 1 - \delta\right) / \hat{c}_l(k')$
5: $\quad$ **return** $1 - \beta c E$ ▷ zero at the true policy
6: **end function**

**Idea**

The last line is the **unit-free Euler residual** of 3.A1, used to grade every solver since Session 3. Here it is not a diagnostic: it *is* the equation.

**Pitfall**

The clipping matters: during the solve an iterate can propose a $k'$ outside the domain, where a Chebyshev polynomial diverges like $\cosh$ (5.B2).

Zhiqiang Feng · Unauthorized distribution prohibited without permission

**A nonlinear solver needs a starting point near the answer. 5.A3's first-order policy, evaluated at the nodes and fitted, is that point.**

The first-order policy of 5.A3 is $\hat{c} = a_k \hat{k} + a_z z$ (its row $C$), or in levels $c = c^* \exp\big(a_k \log(k/k^*) + a_z z_j\big)$. Evaluate it at the $8$ Chebyshev nodes and take the $8$ coefficients that pass through those values:

1: $(a_k, a_z) \leftarrow$ the first-order policy $C$ ▷ 5.A3, Part 1
2: $x_m \leftarrow \cos\frac{(2m-1)\pi}{2(n+1)},\ \ m = 1, \ldots, n+1$ ▷ nodes
3: $k_m \leftarrow k_{lo} + \frac{1}{2}(x_m + 1)(k_{hi} - k_{lo})$
4: **for** $j = 1, \ldots, 7$ **do**
5: $\quad c_m \leftarrow c^* \exp\big(a_k \log(k_m/k^*) + a_z z_j\big)$ for every $m$
6: $\quad \theta_j^0 \leftarrow$ the coefficients that interpolate $(x_m, c_m)$
7: **end for**

The nonlinear solve that follows converges from this guess without trouble.

**Ledger** `worked_proj.warm_start`.

**This is why Block A came first.** Projection needs a good initial guess for a $56$-dimensional nonlinear system, and perturbation produces one in under a millisecond.

5.B1 promised the two methods mix rather than compete. This is where it happens, and it is standard practice, not a trick.

© Zhigang Feng Redistribution prohibited without permission

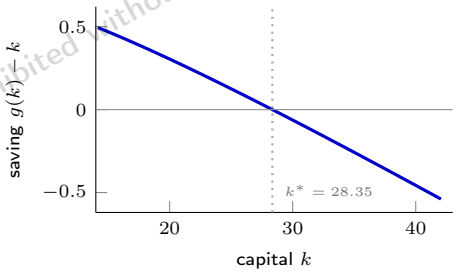
# 3 · Steps 4 and 5: Solve, Then Grade

**Count unknowns against equations, hand the square system to a root-finder, then grade on points you did not solve at.**

**Step 4, solve. Unknowns:** 8 coefficients × 7 states = 56. **Equations:** the residual at the 8 nodes in each of the 7 states = 56. Square, so a root-finder for systems (4.A2) from $\theta^0$:

1: **function** EQUATIONS($\theta$) ▷ 56 in, 56 out
2: **for** $j = 1, \ldots, 7$ and $m = 1, \ldots, 8$ **do**
3: $e_{jm} \leftarrow$ RESIDUAL($\theta, k_m, j$)
4: **end for**
5: **return** all 56 values $e_{jm}$
6: **end function**
7: $\theta \leftarrow$ ROOTFIND(EQUATIONS, $\theta^0$) ▷ tol $10^{-12}$
8: draw 2,000 random points $(k, j)$; report $\max|\mathcal{E}|$ there

**Step 5, grade.** At the nodes the residual is $10^{-15}$ *by construction* and means nothing. Evaluate it somewhere else.




Saving $g(k) - k$ at the middle shock: zero at $k^*$, positive below, negative above, smooth. No grid to quantise it (4.B4).

© Zhiyang Fe… Redistribution prohibited without permissio…

Ledger worked_proj (56 unknowns, 0.13 s), worked_proj_check, fig_proj_policy.

# One Model, Four Methods

**Identical model, chain, domain and residual. The grid method is last on every measure; only collocation is as accurate far from $k^*$ as near it.**

| Method | numbers | sec | $\max\lvert\mathcal{E}\rvert$ near $k^*$ | $\max\lvert\mathcal{E}\rvert$ wide |
|---|---|---|---|---|
| global VFI (4.B4) | 2,100 | 0.94 | $10^{-2.82}$ | $10^{-2.51}$ |
| perturbation, 1st order (5.A3) | **4** | $< 0.001$ | $10^{-4.01}$ | $10^{-2.56}$ |
| perturbation, 2nd order | 12 | 0.02 | $10^{-6.14}$ | $10^{-4.50}$ |
| Chebyshev collocation (5.B4) | 56 | 0.13 | $10^{-6.82}$ | $\mathbf{10^{-6.40}}$ |

The grid stores 37× more numbers than collocation, takes 7× longer, and is **four orders of magnitude** less accurate.

**Pitfall: when the ranking reverses**

This holds for a **smooth, one-dimensional, unconstrained** model. Add a binding constraint: Chebyshev gets Gibbs, perturbation does not exist, VFI does not care (S6).

© Zhigang Feng · Redistribution prohibited without permission

Ledger `three_methods`. **Comparisons of this kind**: **Aruoba, Fernández-Villaverde & Rubio-Ramírez (2006)**, *JEDC* 30.

# How It Works: Steps 1 to 3 in Python

**Steps 1 and 2: basis and residual**

```
def psi(k):                          # [klo,khi] -> [-1,1]
    return 2*(k - klo)/(khi - klo) - 1
def cpol(th, k, j):                  # policy for state j
    return chebval(psi(k), th[j])
def resid(th, k, j):                 # z[j] = exp(z_j)
    c  = cpol(th, k, j)
    y  = z[j]*k**alpha + (1-delta)*k
    kp = np.clip(y - c, klo, khi)
    rhs = 0.0
    for l in range(n_z):
        cp = cpol(th, kp, l)
        R  = alpha*z[l]*kp**(alpha-1) + 1-delta
        rhs += Pi[j,l] * R/cp
    return 1.0 - c*beta*rhs
```

**Step 3: warm start**

```
# first-order policy C (5.A3); bk_solve: Lab 5
C, K, _, _, _ = bk_solve(A, B, n_x=2)
nodes = cos(pi*(2*arange(1,n+2)-1)/(2*(n+1)))
kn = klo + 0.5*(nodes+1)*(khi - klo)
th0 = np.zeros((n_z, n+1))
for j in range(n_z):
    ch = C[0,0]*log(kn/kss) + C[0,1]*logz[j]
    th0[j] = chebfit(psi(kn), css*exp(ch), n)
```

`numpy.polynomial.chebyshev`; **the full code, runnable, is Lab L05, Part 5.**

© Zhigang Feng · Redistribution prohibited without permission

# How It Works: Steps 4 and 5 in Python

```
k_cheb = kn                          # the 8 nodes of Step 3
def equations(flat):
    th = flat.reshape(n_z, n+1)
    return np.concatenate([resid(th, k_cheb, j)
                           for j in range(n_z)])
sol = root(equations, th0.ravel(),
           method="hybr", tol=1e-12)
th = sol.x.reshape(n_z, n+1)
#  56 unknowns, 0.13 s, residual 3.8e-15 at the nodes
E = euler_errors(cpol, th, n_test=2000)
#  graded on 2000 RANDOM points, not the nodes
```

scipy.optimize.root; **ledger** worked_proj, worked_proj_check.

`method="hybr"` is Powell's hybrid method from MINPACK: Newton steps for a system, with a Broyden-updated Jacobian and a trust region that falls back toward steepest descent when a step does not reduce the residual — the safeguards 4.A2 described, packaged.

© Zhigang Feng · Redistribution prohibited without permission

# Takeaways

- **Part 1.** "Small" means $\int w_i R = 0$ for $n+1$ weight functions: a functional equation has become a square nonlinear system, solved by Session 4's Newton.
- **Part 2.** Six weights: least squares (symmetric, ill-conditioned), subdomain (zero on average per piece), moments (avoid), collocation (zero at points; where you look is the design), orthogonal collocation (points at the roots of $T_{n+1}$), Galerkin (orthogonal to the basis; accurate, hard to code). On one model and one basis they land within an order of magnitude of each other, and **orthogonal collocation** is best and cheapest.
- **Part 3.** The growth model a fourth way: an $8$-term Chebyshev polynomial per shock state, $56$ unknowns and $56$ equations, warm-started from 5.A3's first-order solution. **Grade on points you did not solve at.**
- **Part 4.** Against the grid solver, collocation stores $37\times$ fewer numbers and is four orders of magnitude more accurate — for a smooth, one-dimensional, unconstrained model. Add a constraint and the ranking reverses.

© Yiyang Feng Redistribution prohibited without permission

# Bridge: The Models We Actually Care About

Session 5 ends with the best solution of the growth model this course will produce by classical means. It also ends with the condition that made it possible: **smooth, one-dimensional, unconstrained**.

Session 2's models are none of those. Aiyagari households sit at a borrowing limit; Krusell–Smith carries a distribution as a state; the life-cycle model has eighty cohorts. Every method in Sessions 3 to 5 has now met the case that defeats it. Next week we take these tools to those models and find out exactly where they stop.

**Next (Session 6)**

Heterogeneous agents I and II: the Aiyagari economy end to end, by value function iteration and by the Euler equation; Krusell–Smith; transitions after an MIT shock, and the life cycle; and the wall every classical method hits.

**Homework 1** is due next Thursday, Oct 29; **Homework 2** (Part A) goes out the same day. Its Q2 benchmarks a neural network against the Chebyshev solve you just saw.

© Zhigang Feng · Redistribution prohibited without permission